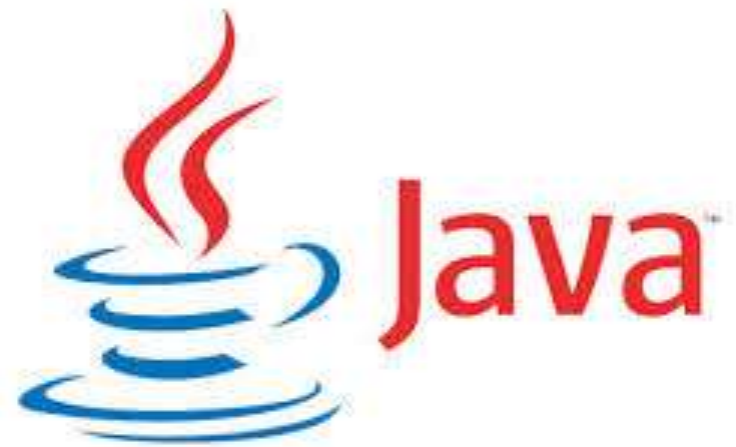




SOC2030

Application Programming in Java



Week 5 Exceptions

Exception handling enables a program to deal with exceptional situations and continue its normal execution.

Dr. Andrei Dragunov

Details

- Why Exceptions
- Standard Exception Handling Options
- Exception Class Hierarchy
- Checked vs. Unchecked Exceptions
- Catching an Exception: try and catch blocks
- Methods Which Throw Exceptions: the throws clause
- Handling vs. Declaring Exceptions
- System Exceptions vs. Application Exceptions
- Writing Custom Exceptions

Exception-Handling Overview

- Exceptions are thrown from a method. The caller of the method can catch and handle the exception.*

```
public static void main(String[] args) {  
    Scanner in = new Scanner(System.in);  
    int a;  
    int b;  
    System.out.print("Enter two integers: ");  
    a = in.nextInt();  
    b = in.nextInt();  
    System.out.println("a / b = " + a/b );  
}
```

Enter two integers: 5 2

a / b = 2

BUILD SUCCESSFUL (total time: 6 seconds)

Enter two integers: 5 0

Exception in thread "main" java.lang.ArithmeticException: / by zero
 at firstexample.FirstExample.main([FirstExample.java:26](#))

Java Result: 1

```
public static void main(String[] args) {  
    Scanner in = new Scanner(System.in);  
    int[] a = {1,2,3,4,5};  
    int i;  
    System.out.print("Enter index: ");  
    i = in.nextInt();  
    System.out.println("a["+i+"] = " + a[i] );  
}
```

Enter index: 2

a[2] = 3

BUILD SUCCESSFUL (total time: 2 seconds)

Enter index: 8

Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException: 8
| at firstexample.FirstExample.main([FirstExample.java:25](#))

Java Result: 1

How to fix?

A simple way to fix this error is to add an **if** statement to test the second number or index:

```
public static void main(String[] args) {  
    Scanner in = new Scanner(System.in);  
    int[] a = {1,2,3,4,5};  
    int i;  
    System.out.print("Enter index: ");  
    i = in.nextInt();  
    if ((i < 5) & (i > 0))  
        System.out.println("a["+i+"] = " + a[i] );  
    else  
        System.out.println("Wring value for i");  
}  
Enter index: 8  
Wring value for i  
BUILD SUCCESSFUL (total time: 2 seconds)
```

The same, but using method `valueAt(int i)`

```
public static int valueAt(int i) {  
    int[] a = {1,2,3,4,5};  
    if ((i >= 5) | (i < 0)) {  
        System.out.println("Wring value for i");  
        System.exit(1);  
    }  
    return a[i];  
}
```



Execution terminates.
It is a clear problem

```
public static void main(String[] args) {  
    Scanner in = new Scanner(System.in);  
    int i;  
    System.out.print("Enter index: ");  
    i = in.nextInt();  
    System.out.println("a["+i+"] = "+ valueAt(i));  
}
```

```
public static int valueAt(int i) {  
    int[] a = {1,2,3,4,5};  
    if ((i >= 5) | (i < 0)) {  
        throw new ArrayIndexOutOfBoundsException("Hello!")  
    }  
    return a[i];  
}
```



Throw exception

```
public static void main(String[] args) {  
    Scanner in = new Scanner(System.in);  
    int i;  
    System.out.print("Enter index: ");  
    i = in.nextInt();  
    try{  
        System.out.println("a["+i+"] = "+ valueAt(i));  
    } catch (ArrayIndexOutOfBoundsException a) {  
        System.out.println(a);  
    }  
    System.out.println("Execution continues ...");  
}
```



No problem

ArrayIndexOutOfBoundsException

The value thrown, in this case **new ArrayIndexOutOfBoundsException("Hello! I'm an exception");**, is called an *exception*. The execution of a **throw** statement is called *throwing an exception*.

The exception is an object created from an exception class. In this case, the exception class is

java.lang. ArrayIndexOutOfBoundsException. The constructor **ArrayIndexOutOfBoundsException(str)** is invoked to construct an exception object, where **str** is a message that describes the exception.

```
try{ ... } catch(...) { ... }
```

When an exception is thrown, the normal execution flow is interrupted. As the name suggests, to “throw an exception” is to pass the exception from one place to another. The statement for invoking the method is contained in a **try** block and a **catch** block. The **try** block contains the code that is executed in normal circumstances. The exception is caught by the **catch** block. The code in the **catch** block is executed to *handle the exception*.

Afterward, the statement after the **catch** block is executed.

The **throw** statement is analogous to a method call, but instead of calling a method, it calls a **catch** block. In this sense, a **catch** block is like a method definition with a parameter that matches the type of the value being thrown. Unlike a method, however, after the **catch** block is executed, the program control does not return to the **throw** statement; instead, it executes the next statement after the **catch** block.

The identifier **a** in the **catch**–block header

catch(ArrayIndexOutOfBoundsException a)

acts very much like a parameter in a method. Thus, this parameter is referred to as a **catch**–block parameter. The type (e.g.,

ArrayIndexOutOfBoundsException) preceding **a** specifies what kind of exception the **catch** block can catch. Once the exception is caught, you can access the thrown value from this parameter in the body of a **catch** block.

In summary, a template for a **try-throw-catch** block may look like this:

```
try {  
    Code to run;  
    A statement or a method that may throw an exception;  
    More code to run;  
}  
catch (type ex) {  
    Code to process the exception;  
}
```

- An exception may be thrown directly by using a **throw** statement in a **try** block, or by invoking a method that may throw an exception.

Advantage of using exception handling

Now you can see the *advantage* of using exception handling: It enables a method to throw an exception to its caller, enabling the caller to handle the exception. Without this capability, the called method itself must handle the exception or terminate the program. Often the called method does not know what to do in case of error. This is typically the case for the library methods. The library method can detect the error, but only the caller knows what needs to be done when an error occurs. The key benefit of exception handling is separating the detection of an error (done in a called method) from the handling of an error (done in the calling method).

What is the output of the following code?

```
public static void main(String[] args) {  
    try {  
        int value = 30;  
        if (value < 40)  
            throw new Exception("value is too small");  
    }  
    catch (Exception ex) {  
        System.out.println(ex.getMessage());  
    }  
    System.out.println("Continue after the catch block");  
}
```

And now?

```
public static void main(String[] args) {  
    try {  
        int value = 50;  
        if (value < 40)  
            throw new Exception("value is too small");  
    }  
    catch (Exception ex) {  
        System.out.println(ex.getMessage());  
    }  
    System.out.println("Continue after the catch block");  
}
```

Show the output of the following code

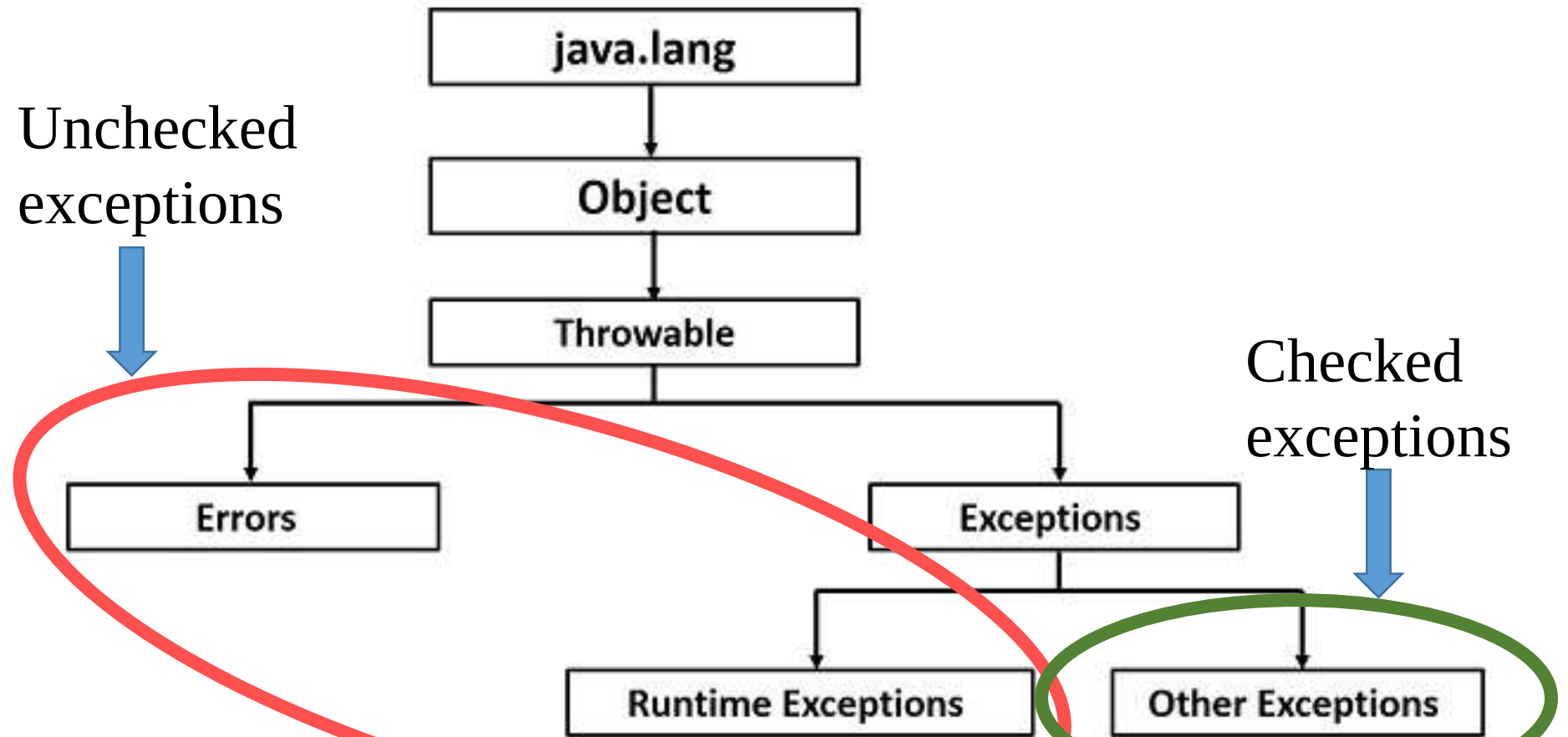
```
public static void main(String[] args) {  
    try {  
        for (int i = 0; i < 2; i++)  
        {  
            System.out.print(i + " ");  
            System.out.println(i/0);  
        }  
    }  
    catch (Exception ex) {  
    }  
}
```

Exception Types

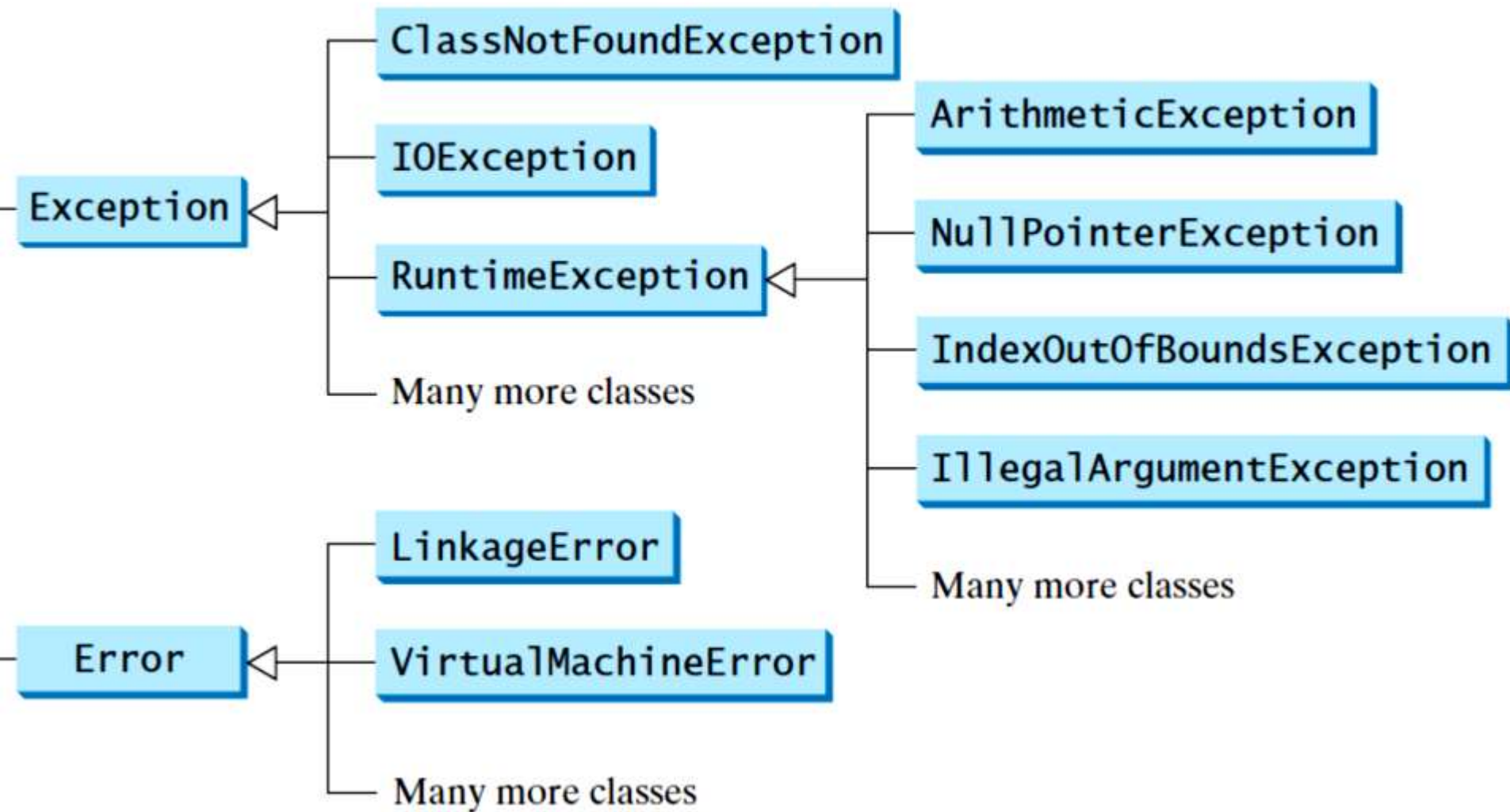
*Exceptions are objects, and objects are defined using classes. The root class for exceptions is **java.lang.Throwable**.*

The preceding section used the classes **ArithmeticException** and **ArrayIndexOutOfBoundsException**. Are there any other types of exceptions you can use? Also you define your own exception classes.

Exceptions hierarchy



The exception classes can be classified into three major types: system errors, exceptions, and runtime exceptions.



Checked and Unchecked exceptions

RuntimeException, **Error**, and their subclasses are known as ***unchecked exceptions***.

All other exceptions are known as ***checked exceptions***, meaning that the compiler forces the programmer to check and deal with them in a **try-catch** block or declare it in the method header.

Examples of Subclasses of Error

<i>Class</i>
LinkageError
VirtualMachineError

Reasons for Exception

A class has some dependency on another class, but the latter class has changed incompatibly after the compilation of the former class.

The JVM is broken or has run out of the resources it needs in order to continue operating.

Examples of Subclasses of Exception

Class

ClassNotFoundException

IOException

Reasons for Exception

Attempt to use a class that does not exist. This exception would occur, for example, if you tried to run a nonexistent class using the **java** command, or if your program were composed of, say, three class files, only two of which could be found.

Related to input/output operations, such as invalid input, reading past the end of a file, and opening a nonexistent file. Examples of subclasses of **IOException** are **InterruptedIOException**, **EOFException** (EOF is short for End of File), and **FileNotFoundException**.

Runtime exceptions are represented in the **RuntimeException** class, which describes programming errors, such as bad casting, accessing an out-of-bounds array, and numeric errors.

Class

ArithmeticException

NullPointerException

IndexOutOfBoundsException

IllegalArgumentException

Reasons for Exception

Dividing an integer by zero. Note that floating-point arithmetic does not throw exceptions (see Appendix E, Special Floating-Point Values).

Attempt to access an object through a **null** reference variable.

Index to an array is out of range.

A method is passed an argument that is illegal or inappropriate.

In most cases, unchecked exceptions reflect programming logic errors that are unrecoverable. For example, a **NullPointerException** is thrown if you access an object through a reference variable before an object is assigned to it; an **IndexOutOfBoundsException** is thrown if you access an element in an array outside the bounds of the array. These are logic errors that should be corrected in the program. Unchecked exceptions can occur anywhere in a program. To avoid cumbersome overuse of **try-catch** blocks, Java does not mandate that you write code to catch or declare unchecked exceptions.

More on Exception Handling

Java's exception-handling model is based on three operations:

- ❑ *declaring an exception*
- ❑ *throwing an exception*
- ❑ *catching an exception*

```
method2() throws Exception {
```

```
    if (an error occurs) {
```

```
        throw new Exception();
```

```
    }
```

```
}
```

Declare exception

Throw exception

Catch exception

```
method1() {
```

```
    try {
```

```
        invoke method2;
```

```
    }
```

```
    catch (Exception ex) {
```

```
        Process exception;
```

```
    }
```

```
}
```

Declaring Exceptions

In Java, the statement currently being executed belongs to a method. The Java interpreter invokes the **main** method to start executing a program.

Every method must state the types of checked exceptions it might throw.

This is known as ***declaring exceptions***.

Because system errors and runtime errors can happen to any code, Java does not require that you declare **Error** and **RuntimeException** (unchecked exceptions) explicitly in the method. However, all other exceptions thrown by the method must be explicitly declared in the method header so that the caller of the method is informed of the exception.

To declare an exception in a method, use the **throws** keyword in the method header, as in this example:

```
public void myMethod() throws IOException
```

The **throws** keyword indicates that **myMethod** might throw an **IOException**. If the method might throw multiple exceptions, add a list of the exceptions, separated by commas, after **throws**:

```
public void myMethod()  
    throws Exception1, Exception2, ..., ExceptionN
```

Throwing Exceptions

A program that detects an error can create an instance of an appropriate exception type and throw it. This is known as *throwing an exception*.

```
IllegalArgumentException ex =  
    new IllegalArgumentException("Wrong Argument");  
throw ex;
```

Or, if you prefer, you can use the following:

```
throw new IllegalArgumentException("Wrong Argument");
```

Catching Exceptions

When an exception is thrown, it can be caught and handled in a **try-catch** block, as follows:

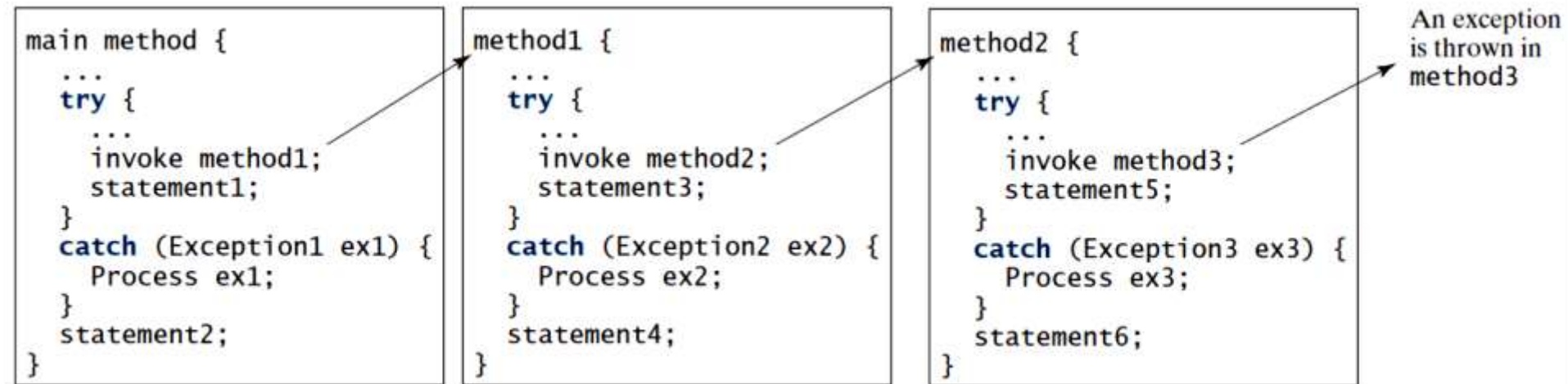
```
try { // Statements that may throw exceptions
    statements;
}
catch (Exception1 exVar1) {
    handler for exception1;
}
catch (Exception2 exVar2) {
    handler for exception2;
}
...
catch (ExceptionN exVarN) {
    handler for exceptionN;
}
```

Catching Exceptions

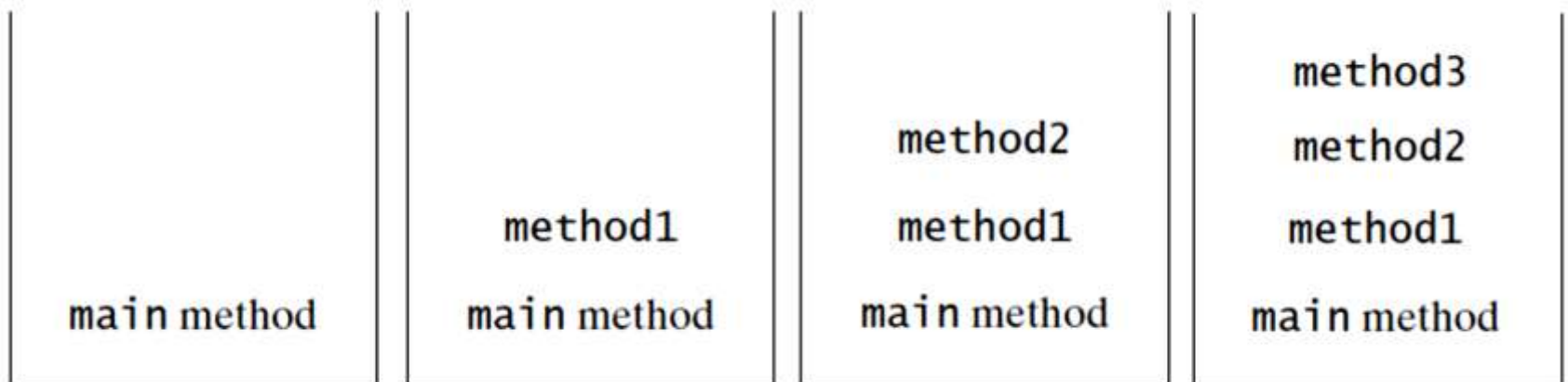
If no exceptions arise during the execution of the **try** block, the **catch** blocks are skipped.

If one of the statements inside the **try** block throws an exception, Java skips the remaining statements in the **try** block and starts the process of finding the code to handle the exception. The code that handles the exception is called the *exception handler*; it is found by *propagating the exception* backward through a chain of method calls, starting from the current method. Each **catch** block is examined in turn, from first to last, to see whether the type of the exception object is an instance of the exception class in the **catch** block.

Suppose the **main** method invokes **method1**, **method1** invokes **method2**, **method2** invokes **method3**, and **method3** throws an exception, as shown in Figure



Call stack



- If the exception type is **Exception3**, it is caught by the **catch** block for handling exception **ex3** in **method2**. **statement5** is skipped, and **statement6** is executed.
- If the exception type is **Exception2**, **method2** is aborted, the control is returned to **method1**, and the exception is caught by the **catch** block for handling exception **ex2** in **method1**. **statement3** is skipped, and **statement4** is executed.
- If the exception type is **Exception1**, **method1** is aborted, the control is returned to the **main** method, and the exception is caught by the **catch** block for handling exception **ex1** in the **main** method. **statement1** is skipped, and **statement2** is executed.
- If the exception type is not caught in **method2**, **method1**, or **main**, the program terminates, and **statement1** and **statement2** are not executed.

Note

The order in which exceptions are specified in **catch** blocks is important. A compile error will result if a catch block for a superclass type appears before a catch block for a subclass type.

RuntimeException is a subclass of **Exception**. The correct ordering should be as shown.

```
try {  
    ...  
}  
catch (Exception ex) {  
    ...  
}  
catch (RuntimeException ex) {  
    ...  
}
```

(a) Wrong order

```
try {  
    ...  
}  
catch (RuntimeException ex) {  
    ...  
}  
catch (Exception ex) {  
    ...  
}
```

(b) Correct order

Java forces you to deal with checked exceptions. If a method declares a checked exception, you must invoke it in a **try-catch** block or declare to throw the exception in the calling method. For example, suppose that method **p1** invokes method **p2**, and **p2** may throw a checked exception (e.g., **IOException**); you have to write the code as shown in (a) or (b) below.

```
void p1() {  
    try {  
        p2();  
    }  
    catch (IOException ex) {  
        ...  
    }  
}
```

(a) Catch exception

```
void p1() throws IOException {  
    p2();  
}
```

(b) Throw exception

Getting Information from Exceptions

An exception object contains valuable information about the exception. You may use the following instance methods in the **java.lang.Throwable** class to get information regarding the exception

java.lang.Throwable

+getMessage(): String

+toString(): String

+printStackTrace(): void

+getStackTrace()
Stack Trace

Returns the message that describes this exception object.

Returns the concatenation of three strings: (1) the full name of the exception class; (2) ":" (a colon and a space); (3) the **getMessage()** method.

Prints the **Throwable** object and its call stack trace information on the console.

Returns an array of stack trace elements representing the stack trace pertaining to this exception object.

The **finally** Clause

*The **finally** clause is always executed regardless whether an exception occurred or not.*

```
try {  
    statements;  
}  
catch (TheException ex) {  
    handling ex;  
}  
  
finally {  
    finalStatements;  
}
```

The code in the **finally** block is executed under all circumstances, regardless of whether an exception occurs in the **try** block or is caught.

Defining Custom Exception Classes

*You can define a custom exception class by extending the **java.lang.Exception** class.*

```
public class InvalidRadiusException extends Exception {  
    private double radius;  
  
    /** Construct an exception */  
    public InvalidRadiusException(double radius) {  
        super("Invalid radius " + radius);  
        this.radius = radius;  
    }  
  
    /** Return the radius */  
    public double getRadius() {  
        return radius;  
    }  
}
```